SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI – 602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCHEMA ANALYSIS AND VIVA VOCE

Course Code:	DSA05	**Course Title:**	Query Processing for Data Science With Real Time Applications
CO Assessed:	CO2	**Assessment:**	Tool 2 – Schema Analysis and Viva Voce
Weightage:		**Total Marks:**	20 Marks
CO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BTL6)			
Student Name:	G Rajasekhar Reddy	**Reg. No.:**	192424216
Section / Batch:	BTech-AI&DS	**Date of Submission:**	14-07-2026
Faculty In-charge:	Dr. B. Shyamala Gowri	**Project Mode:**	Individual Submission

Code of Conduct

I G .Rajasekhar Reddy(192424216) (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: G. Rajasekhar Reddy

Requirement Analysis (4 Marks)	ER Diagram Design (4 Marks)	Table Design & Normalization (4 Marks)	Database Connectivity using Python (4 Marks)	CRUD Operations (4 Marks)	Total (20 Marks)

Scenario 1: University Student Management System

A university manages information related to students, faculty members, departments, courses, and examinations. Currently, student information is stored in spreadsheets by different departments, resulting in duplicated records, inconsistent student IDs, and difficulties in generating reports. The university administration wants to develop a centralized database system that can store student details, course enrollments, faculty assignments, attendance records, and examination results. Students should be able to register for multiple courses, while each course may have multiple students enrolled. Faculty members may teach several courses within a semester. The management also requires reports such as department-wise student strength, faculty workload, and student performance statistics. Design an appropriate database schema by identifying entities, attributes, primary keys, foreign keys, and relationships. Connect the database using Python and demonstrate CRUD operations for managing student records.

Code:

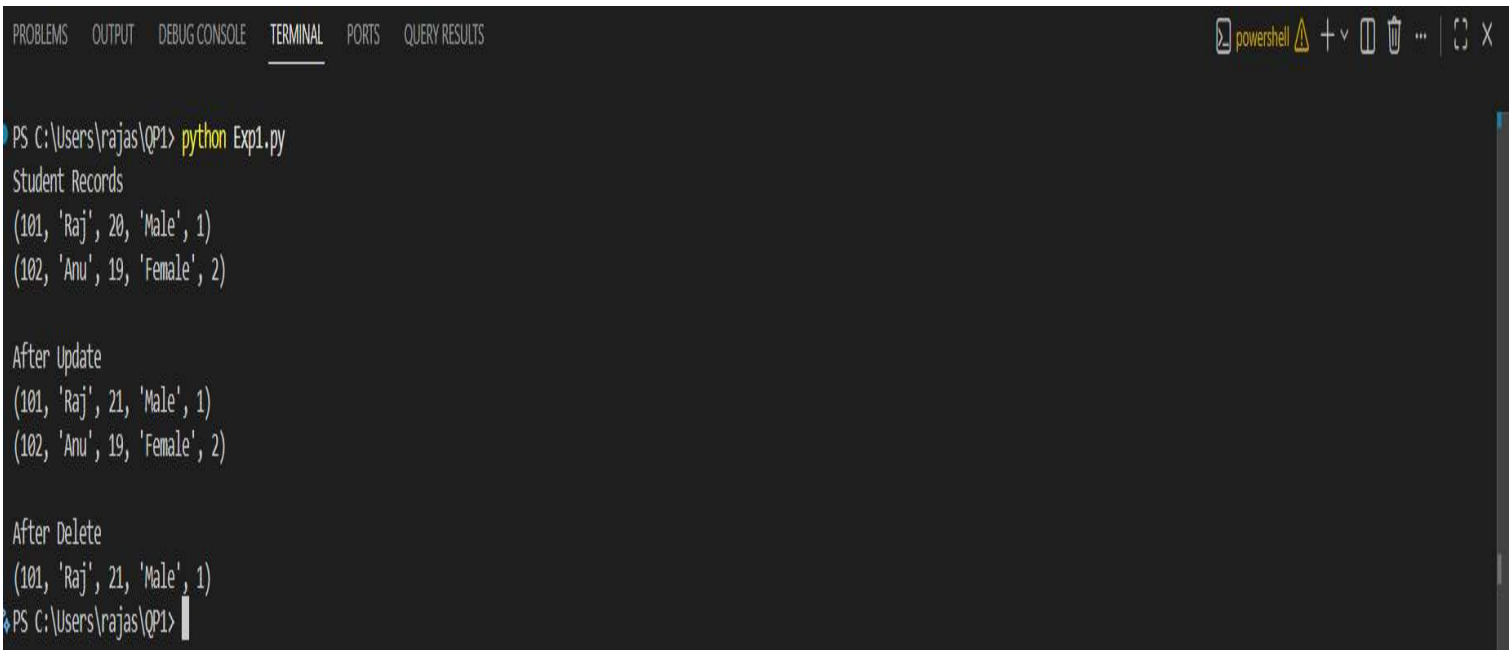
```
import sqlite3

conn = sqlite3.connect("university.db")
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS Department(
DepartmentID INTEGER PRIMARY KEY,
DepartmentName TEXT
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Student(
StudentID INTEGER PRIMARY KEY,
Name TEXT,
Age INTEGER,
Gender TEXT,
DepartmentID INTEGER,
FOREIGN KEY(DepartmentID) REFERENCES Department(DepartmentID)
```

```
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Faculty(
FacultyID INTEGER PRIMARY KEY,
FacultyName TEXT,
DepartmentID INTEGER,
FOREIGN KEY(DepartmentID) REFERENCES Department(DepartmentID)
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Course(
CourseID INTEGER PRIMARY KEY,
CourseName TEXT,
FacultyID INTEGER,
FOREIGN KEY(FacultyID) REFERENCES Faculty(FacultyID)
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Enrollment(
EnrollmentID INTEGER PRIMARY KEY,
StudentID INTEGER,
CourseID INTEGER,
FOREIGN KEY(StudentID) REFERENCES Student(StudentID),
FOREIGN KEY(CourseID) REFERENCES Course(CourseID)
)
""")

cursor.execute("INSERT INTO Department VALUES(1,'AI&DS')")
cursor.execute("INSERT INTO Department VALUES(2,'CSE')")
```

```
cursor.execute("INSERT INTO Student VALUES(101,'Raj',20,'Male',1)")
cursor.execute("INSERT INTO Student VALUES(102,'Anu',19,'Female',2)")

cursor.execute("INSERT INTO Faculty VALUES(201,'Dr.Ravi',1)")
cursor.execute("INSERT INTO Faculty VALUES(202,'Dr.Sita',2)")

cursor.execute("INSERT INTO Course VALUES(301,'Python',201)")
cursor.execute("INSERT INTO Course VALUES(302,'DBMS',202)")

cursor.execute("INSERT INTO Enrollment VALUES(1,101,301)")
cursor.execute("INSERT INTO Enrollment VALUES(2,102,302)")

conn.commit()

print("Student Records")

cursor.execute("SELECT * FROM Student")
rows = cursor.fetchall()

for row in rows:
    print(row)

cursor.execute("""
UPDATE Student
SET Age=21
WHERE StudentID=101
""")

conn.commit()

print("\nAfter Update")
```

```
cursor.execute("SELECT * FROM Student")

for row in cursor.fetchall():
    print(row)

cursor.execute("""
DELETE FROM Student
WHERE StudentID=102
""")

conn.commit()

print("\nAfter Delete")

cursor.execute("SELECT * FROM Student")

for row in cursor.fetchall():
    print(row)

conn.close()
```

Output:



Scenario 2: Hospital Patient Record Management System

A multi-specialty hospital receives thousands of patients every month. Currently, patient records are maintained manually, making it difficult to track patient history, doctor consultations, prescriptions, laboratory tests, and billing information. The hospital management wants a database-driven application that can store patient details, doctor information, appointments, treatment records, and payment details. Each patient may visit multiple doctors over time, and doctors may consult many patients. Laboratory tests and prescriptions must also be linked to patient records. The system should provide quick access to patient histories and generate reports on disease trends and doctor workloads. Design a relational database schema for this system and use Python SQL libraries to connect to the database and manage patient information efficiently.

Code:

```
import mysql.connector
from mysql.connector import Error

try:
    # Connect to MySQL
    conn = mysql.connector.connect(
        host="127.0.0.1",
        user="root",
        password="",
        database="hospital_db",
        port=3306
    )

    if conn.is_connected():
        print("Connected to MySQL Successfully")

    cursor = conn.cursor()

    # Create Patient Table
    cursor.execute("""
```

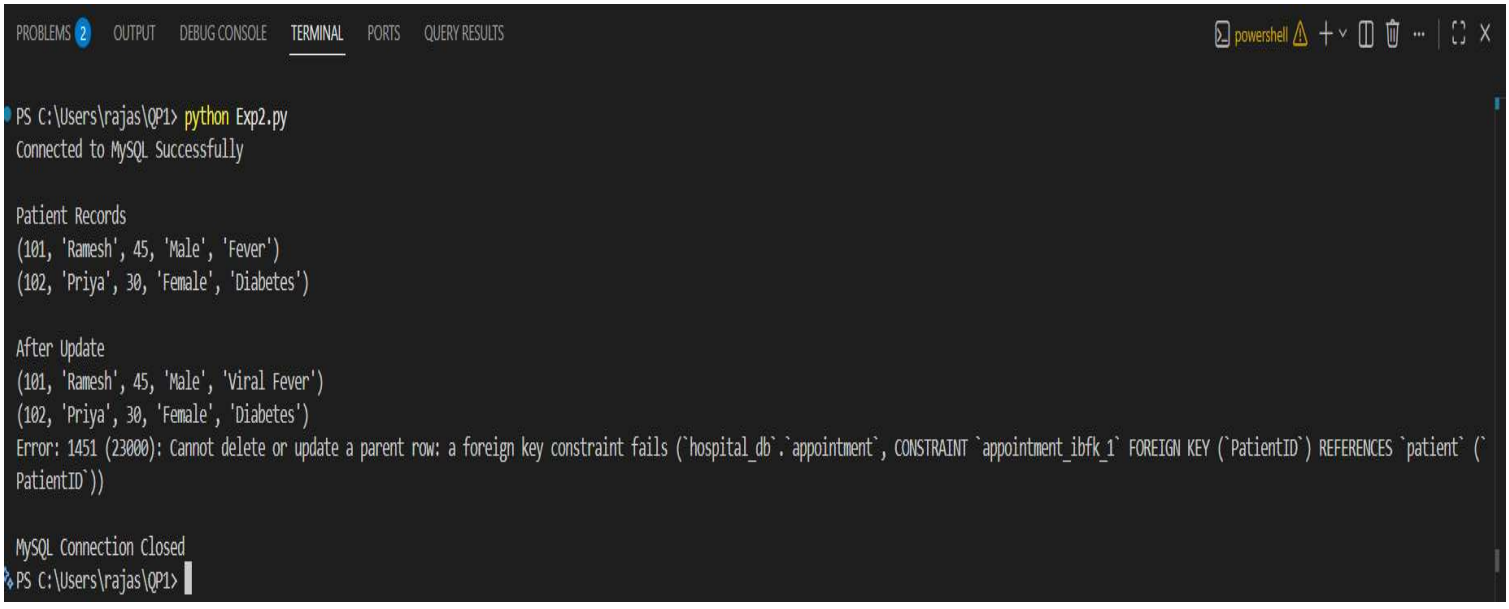
```
CREATE TABLE IF NOT EXISTS Patient(
    PatientID INT PRIMARY KEY,
    PatientName VARCHAR(50),
    Age INT,
    Gender VARCHAR(10),
    Disease VARCHAR(50)
)
""")

# Create Doctor Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Doctor(
    DoctorID INT PRIMARY KEY,
    DoctorName VARCHAR(50),
    Specialization VARCHAR(50)
)
""")

# Create Appointment Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Appointment(
    AppointmentID INT PRIMARY KEY,
    PatientID INT,
    DoctorID INT,
    AppointmentDate DATE,
    FOREIGN KEY (PatientID) REFERENCES Patient(PatientID),
    FOREIGN KEY (DoctorID) REFERENCES Doctor(DoctorID)
)
""")

# Create Treatment Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Treatment(
```

```
    TreatmentID INT PRIMARY KEY,
    PatientID INT,
    Prescription VARCHAR(100),
    LabTest VARCHAR(100),
    FOREIGN KEY (PatientID) REFERENCES Patient(PatientID)
)
""")

# Create Payment Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Payment(
    PaymentID INT PRIMARY KEY,
    PatientID INT,
    Amount DECIMAL(10,2),
    PaymentStatus VARCHAR(20),
    FOREIGN KEY (PatientID) REFERENCES Patient(PatientID)
)
""")

# Clear old records (prevents duplicate key errors)
cursor.execute("DELETE FROM Payment")
cursor.execute("DELETE FROM Treatment")
cursor.execute("DELETE FROM Appointment")
cursor.execute("DELETE FROM Doctor")
cursor.execute("DELETE FROM Patient")

# Insert Patient
cursor.execute("INSERT INTO Patient VALUES (101,'Ramesh',45,'Male','Fever')")
cursor.execute("INSERT INTO Patient VALUES (102,'Priya',30,'Female','Diabetes')")

# Insert Doctor
cursor.execute("INSERT INTO Doctor VALUES (201,'Dr.Kumar','General')")
cursor.execute("INSERT INTO Doctor VALUES (202,'Dr.Anita','Diabetologist')")
```

```
# Insert Appointment
cursor.execute("INSERT INTO Appointment VALUES (1,101,201,'2026-07-14')")
cursor.execute("INSERT INTO Appointment VALUES (2,102,202,'2026-07-15')")

# Insert Treatment
cursor.execute("INSERT INTO Treatment VALUES (1,101,'Paracetamol','Blood Test')")
cursor.execute("INSERT INTO Treatment VALUES (2,102,'Insulin','Sugar Test')")

# Insert Payment
cursor.execute("INSERT INTO Payment VALUES (1,101,500.00,'Paid')")
cursor.execute("INSERT INTO Payment VALUES (2,102,1200.00,'Pending')")

conn.commit()

print("\nPatient Records")
cursor.execute("SELECT * FROM Patient")

for row in cursor.fetchall():
    print(row)

# Update
cursor.execute("""
UPDATE Patient
SET Disease='Viral Fever'
WHERE PatientID=101
""")

conn.commit()

print("\nAfter Update")
cursor.execute("SELECT * FROM Patient")
```

```
        for row in cursor.fetchall():
            print(row)

        # Delete
        cursor.execute("DELETE FROM Patient WHERE PatientID=102")
        conn.commit()

        print("\nAfter Delete")
        cursor.execute("SELECT * FROM Patient")

        for row in cursor.fetchall():
            print(row)

    except Error as e:
        print("Error:", e)

    finally:
        if 'conn' in locals() and conn.is_connected():
            cursor.close()
            conn.close()
            print("\nMySQL Connection Closed")
```

Output:



Scenario 3: E-Commerce Online Shopping Platform

An online shopping company sells products across different categories such as electronics, clothing, books, and household items. Customers create accounts, browse products, place orders, and make online payments. Each order may contain multiple products, and inventory levels must be updated automatically after each purchase. The company also wants to track customer reviews and ratings for products. The management requires reports on top-selling products, customer purchase patterns, and monthly revenue. Analyze the business requirements and design a database containing tables for customers, products, orders, payments, and reviews. Establish relationships among the tables and develop Python programs to perform database connectivity and CRUD operations.

Code:

```
import mysql.connector
from mysql.connector import Error

try:
    conn = mysql.connector.connect(
        host="127.0.0.1",
        user="root",
        password="",
        database="ecommerce_db",
        port=3306
    )

    if conn.is_connected():
        print("Connected to MySQL Successfully")

    cursor = conn.cursor()
```

```
cursor.execute("""
CREATE TABLE IF NOT EXISTS Customer(
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(50),
    Email VARCHAR(50),
    Phone VARCHAR(15)
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Product(
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(50),
    Category VARCHAR(50),
    Price DECIMAL(10,2),
    Stock INT
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Orders(
    OrderID INT PRIMARY KEY,
    CustomerID INT,
    OrderDate DATE,
    TotalAmount DECIMAL(10,2),
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)
)
```

```
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Payment(
   PaymentID INT PRIMARY KEY,
   OrderID INT,
   PaymentMethod VARCHAR(30),
   PaymentStatus VARCHAR(20),
   FOREIGN KEY (OrderID) REFERENCES Orders(OrderID)
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Review(
   ReviewID INT PRIMARY KEY,
   CustomerID INT,
   ProductID INT,
   Rating INT,
   ReviewText VARCHAR(100),
   FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID),
   FOREIGN KEY (ProductID) REFERENCES Product(ProductID)
)
""")

cursor.execute("DELETE FROM Review")
cursor.execute("DELETE FROM Payment")
cursor.execute("DELETE FROM Orders")
cursor.execute("DELETE FROM Product")
```

```
cursor.execute("DELETE FROM Customer")

cursor.execute("INSERT INTO Customer VALUES (101,'Rahul','rahul@gmail.com','9876543210')")
cursor.execute("INSERT INTO Customer VALUES (102,'Priya','priya@gmail.com','9876543211')")

cursor.execute("INSERT INTO Product VALUES (201,'Laptop','Electronics',55000,10)")
cursor.execute("INSERT INTO Product VALUES (202,'Book','Books',500,25)")

cursor.execute("INSERT INTO Orders VALUES (301,101,'2026-07-14',55000)")
cursor.execute("INSERT INTO Orders VALUES (302,102,'2026-07-15',500)")

cursor.execute("INSERT INTO Payment VALUES (401,301,'UPI','Paid')")
cursor.execute("INSERT INTO Payment VALUES (402,302,'Card','Pending')")

cursor.execute("INSERT INTO Review VALUES (501,101,201,5,'Excellent')")
cursor.execute("INSERT INTO Review VALUES (502,102,202,4,'Good Book')")

cursor.execute("UPDATE Product SET Stock=Stock-1 WHERE ProductID=201")
cursor.execute("UPDATE Product SET Stock=Stock-1 WHERE ProductID=202")

conn.commit()

print("\nCustomer Records")
cursor.execute("SELECT * FROM Customer")
for row in cursor.fetchall():
```

```
        print(row)

    cursor.execute("""
    UPDATE Customer
    SET Phone='9999999999'
    WHERE CustomerID=101
    """)
    conn.commit()

    print("\nAfter Update")
    cursor.execute("SELECT * FROM Customer")
    for row in cursor.fetchall():
        print(row)

    cursor.execute("DELETE FROM Customer WHERE CustomerID=102")
    conn.commit()

    print("\nAfter Delete")
    cursor.execute("SELECT * FROM Customer")
    for row in cursor.fetchall():
        print(row)

except Error as e:
    print("Error:", e)

finally:
    if 'conn' in locals() and conn.is_connected():
        cursor.close()
```

```
conn.close()
print("\nMySQL Connection Closed")
```

Output:

Scenario 4: Library Management System

A large academic library manages thousands of books, journals, magazines, and digital resources. Students and faculty members borrow and return books regularly. The current manual system often results in misplaced records and inaccurate tracking of issued books. The library wants to automate its operations by maintaining information about books, authors, publishers, members, and transactions. A member may borrow multiple books, and a book may be issued to different members over time. The system should also calculate fines for overdue books and generate inventory reports. Design an efficient database schema and implement database connectivity using Python to manage book records and member transactions.

Code:

```
import sqlite3

conn = sqlite3.connect("library.db")
cursor = conn.cursor()
```

```
# Create Author Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Author(
    AuthorID INTEGER PRIMARY KEY,
    AuthorName TEXT
)
""")

# Create Publisher Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Publisher(
    PublisherID INTEGER PRIMARY KEY,
    PublisherName TEXT
)
""")

# Create Book Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Book(
    BookID INTEGER PRIMARY KEY,
    BookName TEXT,
    AuthorID INTEGER,
    PublisherID INTEGER,
    Copies INTEGER,
    FOREIGN KEY (AuthorID) REFERENCES Author(AuthorID),
    FOREIGN KEY (PublisherID) REFERENCES Publisher(PublisherID)
)
""")
```

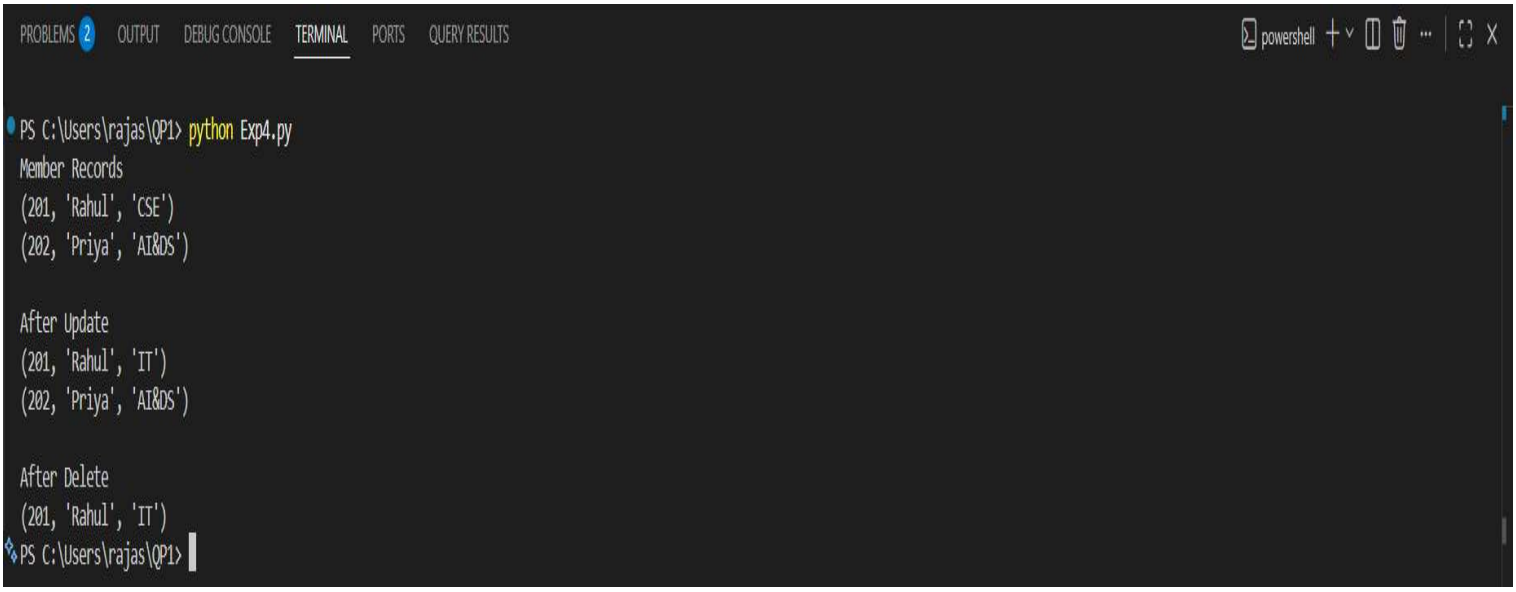
```
# Create Member Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Member(
    MemberID INTEGER PRIMARY KEY,
    MemberName TEXT,
    Department TEXT
)
""")

# Create Transaction Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS TransactionDetails(
    TransactionID INTEGER PRIMARY KEY,
    MemberID INTEGER,
    BookID INTEGER,
    IssueDate TEXT,
    ReturnDate TEXT,
    Fine REAL,
    FOREIGN KEY (MemberID) REFERENCES Member(MemberID),
    FOREIGN KEY (BookID) REFERENCES Book(BookID)
)
""")

# Clear old records
cursor.execute("DELETE FROM TransactionDetails")
cursor.execute("DELETE FROM Member")
cursor.execute("DELETE FROM Book")
```

```
cursor.execute("DELETE FROM Author")
cursor.execute("DELETE FROM Publisher")

# Insert Records
cursor.execute("INSERT INTO Author VALUES(1,'R.K.Narayan')")
cursor.execute("INSERT INTO Author VALUES(2,'Chetan Bhagat')")

cursor.execute("INSERT INTO Publisher VALUES(1,'Pearson')")
cursor.execute("INSERT INTO Publisher VALUES(2,'McGraw Hill')")

cursor.execute("INSERT INTO Book VALUES(101,'Python Programming',1,1,10)")
cursor.execute("INSERT INTO Book VALUES(102,'Database Systems',2,2,8)")

cursor.execute("INSERT INTO Member VALUES(201,'Rahul','CSE')")
cursor.execute("INSERT INTO Member VALUES(202,'Priya','AI&DS')")

cursor.execute("INSERT INTO TransactionDetails VALUES(301,201,101,'2026-07-10','2026-07-15',0)")
cursor.execute("INSERT INTO TransactionDetails VALUES(302,202,102,'2026-07-11','2026-07-18',50)")

# Update available copies after issue
cursor.execute("UPDATE Book SET Copies = Copies - 1 WHERE BookID=101")
cursor.execute("UPDATE Book SET Copies = Copies - 1 WHERE BookID=102")

conn.commit()

# Read
```

```
print("Member Records")
cursor.execute("SELECT * FROM Member")
for row in cursor.fetchall():
    print(row)

# Update
cursor.execute("""
UPDATE Member
SET Department='IT'
WHERE MemberID=201
""")

conn.commit()

print("\nAfter Update")
cursor.execute("SELECT * FROM Member")
for row in cursor.fetchall():
    print(row)

# Delete
cursor.execute("""
DELETE FROM Member
WHERE MemberID=202
""")

conn.commit()

print("\nAfter Delete")
```

```
cursor.execute("SELECT * FROM Member")
for row in cursor.fetchall():
    print(row)

cursor.close()
conn.close()
```

Output:



Scenario 5: Hotel Reservation and Booking System

A hotel chain operates multiple branches across different cities. Customers can book rooms online or through customer service representatives. The hotel management wants a centralized system to manage customer information, room availability, reservations, payments, and employee details. Different room categories such as standard, deluxe, and suite must be maintained. The system should track check-in and check-out details, generate invoices, and maintain customer booking history. Analyze the requirements and design a database structure that minimizes redundancy and supports efficient querying. Use Python to connect to the database and demonstrate room booking, reservation updates, and customer management functionalities.

Code:

```
import mysql.connector
```

```
from mysql.connector import Error

try:
    conn = mysql.connector.connect(
        host="127.0.0.1",
        user="root",
        password="",
        database="hotel_db",
        port=3306
    )

    if conn.is_connected():
        print("Connected to MySQL Successfully")

    cursor = conn.cursor()

    # Customer Table
    cursor.execute("""
    CREATE TABLE IF NOT EXISTS Customer(
        CustomerID INT PRIMARY KEY,
        CustomerName VARCHAR(50),
        Phone VARCHAR(15),
        City VARCHAR(50)
    )
    """)

    # Room Table
    cursor.execute("""
```

```
CREATE TABLE IF NOT EXISTS Room(
    RoomID INT PRIMARY KEY,
    RoomType VARCHAR(30),
    Price DECIMAL(10,2),
    Availability VARCHAR(20)
)
""")

# Employee Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Employee(
    EmployeeID INT PRIMARY KEY,
    EmployeeName VARCHAR(50),
    Designation VARCHAR(50)
)
""")

# Reservation Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Reservation(
    ReservationID INT PRIMARY KEY,
    CustomerID INT,
    RoomID INT,
    CheckIn DATE,
    CheckOut DATE,
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID),
    FOREIGN KEY (RoomID) REFERENCES Room(RoomID)
)
```

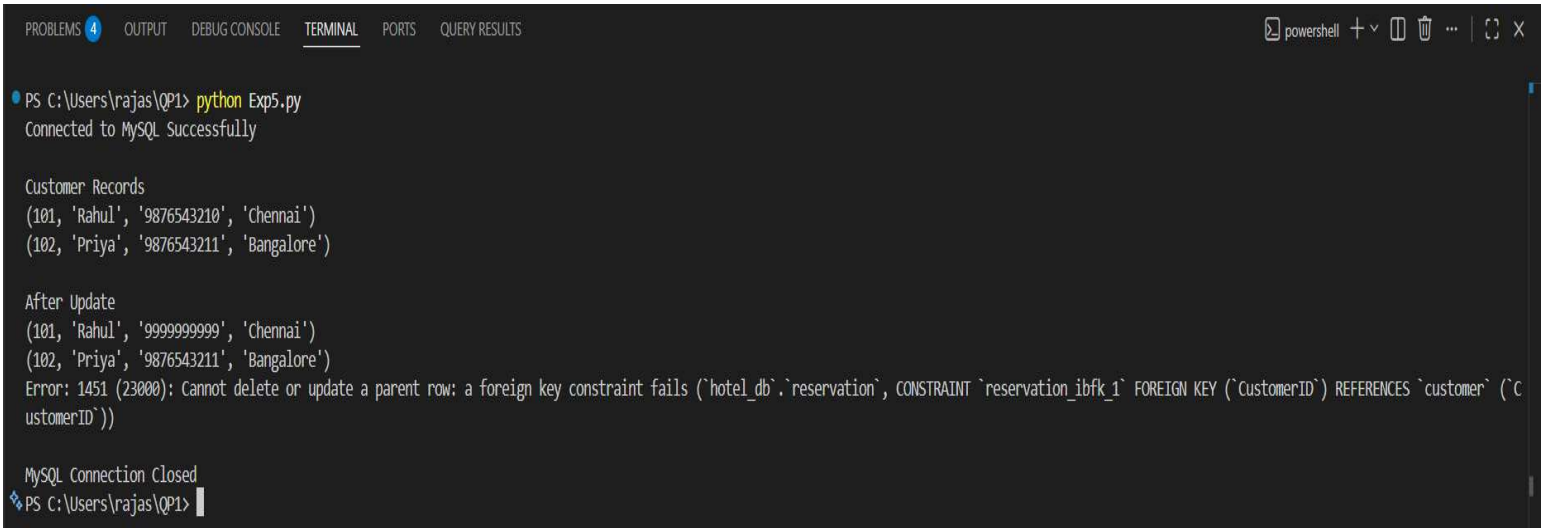
```
    """)

    # Payment Table
    cursor.execute("""
    CREATE TABLE IF NOT EXISTS Payment(
        PaymentID INT PRIMARY KEY,
        ReservationID INT,
        Amount DECIMAL(10,2),
        PaymentStatus VARCHAR(20),
        FOREIGN KEY (ReservationID) REFERENCES Reservation(ReservationID)
    )
    """)

    # Delete Old Records
    cursor.execute("DELETE FROM Payment")
    cursor.execute("DELETE FROM Reservation")
    cursor.execute("DELETE FROM Employee")
    cursor.execute("DELETE FROM Room")
    cursor.execute("DELETE FROM Customer")

    # Insert Customers
    cursor.execute("INSERT INTO Customer VALUES(101,'Rahul','9876543210','Chennai')")
    cursor.execute("INSERT INTO Customer VALUES(102,'Priya','9876543211','Bangalore')")

    # Insert Rooms
    cursor.execute("INSERT INTO Room VALUES(201,'Deluxe',3000,'Available')")
```

```
cursor.execute("INSERT INTO Room VALUES(202,'Suite',5000,'Available')")

# Insert Employees
cursor.execute("INSERT INTO Employee VALUES(301,'Arun','Manager')")
cursor.execute("INSERT INTO Employee VALUES(302,'Meena','Receptionist')")

# Insert Reservations
cursor.execute("INSERT INTO Reservation VALUES(401,101,201,'2026-07-14','2026-07-16')")
cursor.execute("INSERT INTO Reservation VALUES(402,102,202,'2026-07-15','2026-07-18')")

# Insert Payments
cursor.execute("INSERT INTO Payment VALUES(501,401,6000,'Paid')")
cursor.execute("INSERT INTO Payment VALUES(502,402,15000,'Pending')")

# Update Room Availability
cursor.execute("UPDATE Room SET Availability='Booked' WHERE RoomID=201")
cursor.execute("UPDATE Room SET Availability='Booked' WHERE RoomID=202")

conn.commit()

# Read
print("\nCustomer Records")
cursor.execute("SELECT * FROM Customer")
for row in cursor.fetchall():
    print(row)
```

```
    # Update
    cursor.execute("""
    UPDATE Customer
    SET Phone='9999999999'
    WHERE CustomerID=101
    """)

    conn.commit()

    print("\nAfter Update")
    cursor.execute("SELECT * FROM Customer")
    for row in cursor.fetchall():
        print(row)

    # Delete
    cursor.execute("DELETE FROM Customer WHERE CustomerID=102")
    conn.commit()

    print("\nAfter Delete")
    cursor.execute("SELECT * FROM Customer")
    for row in cursor.fetchall():
        print(row)

except Error as e:
    print("Error:", e)

finally:
    if 'conn' in locals() and conn.is_connected():
```

```
    cursor.close()
    conn.close()
    print("\nMySQL Connection Closed")
```

Output:



Scenario 6: Banking and Customer Account Management System

A commercial bank manages savings accounts, current accounts, loans, and customer transactions. Customers may own multiple accounts, and each account contains transaction records such as deposits, withdrawals, and transfers. The bank requires a secure database system to store customer information, account details, branch information, and loan records. Managers need reports on customer balances, transaction histories, and loan repayments. The system must support quick retrieval of information and ensure data consistency. Design the database schema, define relationships between customers and accounts, and implement Python-based database connectivity for account management operations.

Code:

```
import psycopg2

# Connect to PostgreSQL
conn = psycopg2.connect(
    host="localhost",
    database="bank_db",
```

```
    user="postgres",
    password="raju4216",
    port="5432"
)

cursor = conn.cursor()

# Create Branch Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Branch(
    BranchID INT PRIMARY KEY,
    BranchName VARCHAR(50),
    City VARCHAR(50)
)
""")

# Create Customer Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Customer(
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(50),
    Phone VARCHAR(15),
    BranchID INT,
    FOREIGN KEY (BranchID) REFERENCES Branch(BranchID)
)
""")

# Create Account Table
```

```
cursor.execute("""
CREATE TABLE IF NOT EXISTS Account(
    AccountNo INT PRIMARY KEY,
    CustomerID INT,
    AccountType VARCHAR(20),
    Balance DECIMAL(10,2),
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)
)
""")

# Create Transactions Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Transactions(
    TransactionID INT PRIMARY KEY,
    AccountNo INT,
    TransactionType VARCHAR(20),
    Amount DECIMAL(10,2),
    TransactionDate DATE,
    FOREIGN KEY (AccountNo) REFERENCES Account(AccountNo)
)
""")

# Create Loan Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Loan(
    LoanID INT PRIMARY KEY,
    CustomerID INT,
    LoanAmount DECIMAL(10,2),
```

```
    LoanStatus VARCHAR(20),
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID)
)
""")

# Delete old records
cursor.execute("DELETE FROM Transactions")
cursor.execute("DELETE FROM Loan")
cursor.execute("DELETE FROM Account")
cursor.execute("DELETE FROM Customer")
cursor.execute("DELETE FROM Branch")

# Insert Branch
cursor.execute("INSERT INTO Branch VALUES (1,'Main Branch','Chennai')")
cursor.execute("INSERT INTO Branch VALUES (2,'City Branch','Bangalore')")

# Insert Customer
cursor.execute("INSERT INTO Customer VALUES (101,'Rahul','9876543210',1)")
cursor.execute("INSERT INTO Customer VALUES (102,'Priya','9876543211',2)")

# Insert Account
cursor.execute("INSERT INTO Account VALUES (1001,101,'Savings',50000)")
cursor.execute("INSERT INTO Account VALUES (1002,102,'Current',75000)")

# Insert Transactions
cursor.execute("INSERT INTO Transactions VALUES (1,1001,'Deposit',10000,'2026-07-
14')")
```

```
cursor.execute("INSERT INTO Transactions VALUES (2,1002,'Withdrawal',5000,'2026-07-15')")

# Insert Loan
cursor.execute("INSERT INTO Loan VALUES (501,101,200000,'Approved')")
cursor.execute("INSERT INTO Loan VALUES (502,102,150000,'Pending')")

conn.commit()

# Read
print("Customer Records")
cursor.execute("SELECT * FROM Customer")
for row in cursor.fetchall():
    print(row)

# Update
cursor.execute("""
UPDATE Customer
SET Phone='9999999999'
WHERE CustomerID=101
""")
conn.commit()

print("\nAfter Update")
cursor.execute("SELECT * FROM Customer")
for row in cursor.fetchall():
    print(row)
```

```
# Delete (Delete child records first)
cursor.execute("DELETE FROM Transactions WHERE AccountNo=1002")
cursor.execute("DELETE FROM Loan WHERE CustomerID=102")
cursor.execute("DELETE FROM Account WHERE CustomerID=102")
cursor.execute("DELETE FROM Customer WHERE CustomerID=102")

conn.commit()

print("\nAfter Delete")
cursor.execute("SELECT * FROM Customer")
for row in cursor.fetchall():
    print(row)

cursor.close()
conn.close()
```

Output:

Scenario 7: Food Delivery Application

A food delivery platform connects customers, restaurants, and delivery agents through a mobile application. Customers place orders from different restaurants, while delivery agents deliver food to

specified locations. Restaurants maintain menus and pricing information. The platform must track orders, delivery status, payments, and customer feedback. Management wants analytical reports on popular restaurants, peak ordering times, and delivery performance. Design a database containing tables for customers, restaurants, menu items, orders, delivery agents, and payments. Implement Python-based CRUD operations to manage customer orders and delivery records.

Code:

```
import psycopg2

# Connect to PostgreSQL
conn = psycopg2.connect(
    host="localhost",
    database="food_delivery_db",
    user="postgres",
    password="YOUR_PASSWORD",
    port="5432"
)

cursor = conn.cursor()

# Create Customer Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Customer(
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(50),
    Phone VARCHAR(15),
    Address VARCHAR(100)
)
""")
```

```
# Create Restaurant Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Restaurant(
    RestaurantID INT PRIMARY KEY,
    RestaurantName VARCHAR(50),
    Location VARCHAR(50)
)
""")

# Create Menu Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Menu(
    MenuID INT PRIMARY KEY,
    RestaurantID INT,
    ItemName VARCHAR(50),
    Price DECIMAL(10,2),
    FOREIGN KEY (RestaurantID) REFERENCES Restaurant(RestaurantID)
)
""")

# Create Delivery Agent Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS DeliveryAgent(
    AgentID INT PRIMARY KEY,
    AgentName VARCHAR(50),
    Phone VARCHAR(15)
)
""")
```

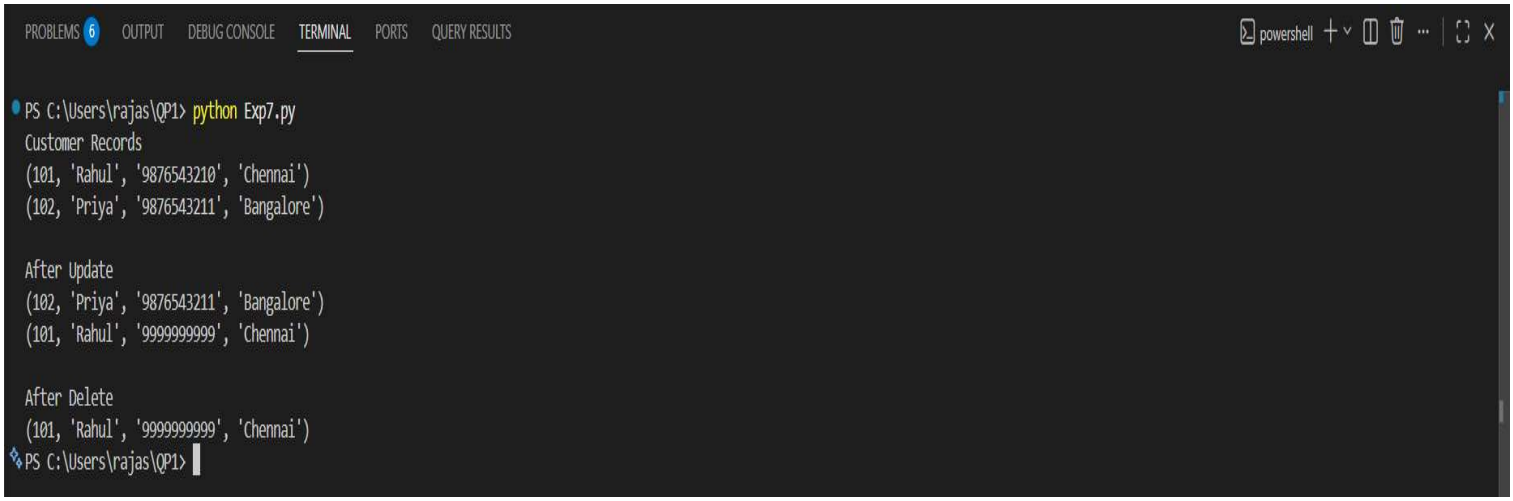
```
# Create Orders Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Orders(
    OrderID INT PRIMARY KEY,
    CustomerID INT,
    RestaurantID INT,
    AgentID INT,
    OrderDate DATE,
    Status VARCHAR(20),
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID),
    FOREIGN KEY (RestaurantID) REFERENCES Restaurant(RestaurantID),
    FOREIGN KEY (AgentID) REFERENCES DeliveryAgent(AgentID)
)
""")

# Create Payment Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Payment(
    PaymentID INT PRIMARY KEY,
    OrderID INT,
    Amount DECIMAL(10,2),
    PaymentStatus VARCHAR(20),
    FOREIGN KEY (OrderID) REFERENCES Orders(OrderID)
)
""")

# Delete Old Records
```

```
cursor.execute("DELETE FROM Payment")
cursor.execute("DELETE FROM Orders")
cursor.execute("DELETE FROM Menu")
cursor.execute("DELETE FROM DeliveryAgent")
cursor.execute("DELETE FROM Restaurant")
cursor.execute("DELETE FROM Customer")

# Insert Customer
cursor.execute("INSERT INTO Customer VALUES (101,'Rahul','9876543210','Chennai')")
cursor.execute("INSERT INTO Customer VALUES (102,'Priya','9876543211','Bangalore')")

# Insert Restaurant
cursor.execute("INSERT INTO Restaurant VALUES (201,'Dominos','Chennai')")
cursor.execute("INSERT INTO Restaurant VALUES (202,'KFC','Bangalore')")

# Insert Menu
cursor.execute("INSERT INTO Menu VALUES (301,201,'Pizza',299)")
cursor.execute("INSERT INTO Menu VALUES (302,202,'Burger',199)")

# Insert Delivery Agent
cursor.execute("INSERT INTO DeliveryAgent VALUES (401,'Arun','9876500000')")
cursor.execute("INSERT INTO DeliveryAgent VALUES (402,'Kumar','9876511111')")

# Insert Orders
cursor.execute("INSERT INTO Orders VALUES (501,101,201,401,'2026-07-14','Delivered')")
```

```
cursor.execute("INSERT INTO Orders VALUES (502,102,202,402,'2026-07-15','Pending')")

# Insert Payment
cursor.execute("INSERT INTO Payment VALUES (601,501,299,'Paid')")
cursor.execute("INSERT INTO Payment VALUES (602,502,199,'Pending')")

conn.commit()

# Read
print("Customer Records")
cursor.execute("SELECT * FROM Customer")
for row in cursor.fetchall():
    print(row)

# Update
cursor.execute("""
UPDATE Customer
SET Phone='9999999999'
WHERE CustomerID=101
""")
conn.commit()

print("\nAfter Update")
cursor.execute("SELECT * FROM Customer")
for row in cursor.fetchall():
    print(row)
```

```
# Delete (Delete child records first)
cursor.execute("DELETE FROM Payment WHERE OrderID=502")
cursor.execute("DELETE FROM Orders WHERE CustomerID=102")
cursor.execute("DELETE FROM Customer WHERE CustomerID=102")
conn.commit()

print("\nAfter Delete")
cursor.execute("SELECT * FROM Customer")
for row in cursor.fetchall():
    print(row)

cursor.close()
conn.close()
```

Output:



Scenario 8: Employee Payroll Management System

A manufacturing company employs hundreds of workers and administrative staff. The Human Resources department currently manages employee information and salary calculations using spreadsheets, which often leads to payroll errors. The company wants a database-driven payroll system to store employee details, department information, attendance records, salary structures, and tax deductions.

Each employee belongs to a department, and payroll must be generated monthly based on attendance and salary components. Design a normalized database schema and create a Python application that performs employee management and payroll processing using database connectivity.

Code:

```
import sqlite3

conn = sqlite3.connect("payroll.db")
cursor = conn.cursor()

# Create Department Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Department(
    DepartmentID INTEGER PRIMARY KEY,
    DepartmentName TEXT
)
""")

# Create Employee Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Employee(
    EmployeeID INTEGER PRIMARY KEY,
    EmployeeName TEXT,
    Designation TEXT,
    DepartmentID INTEGER,
    FOREIGN KEY(DepartmentID) REFERENCES Department(DepartmentID)
)
""")

# Create Attendance Table
```

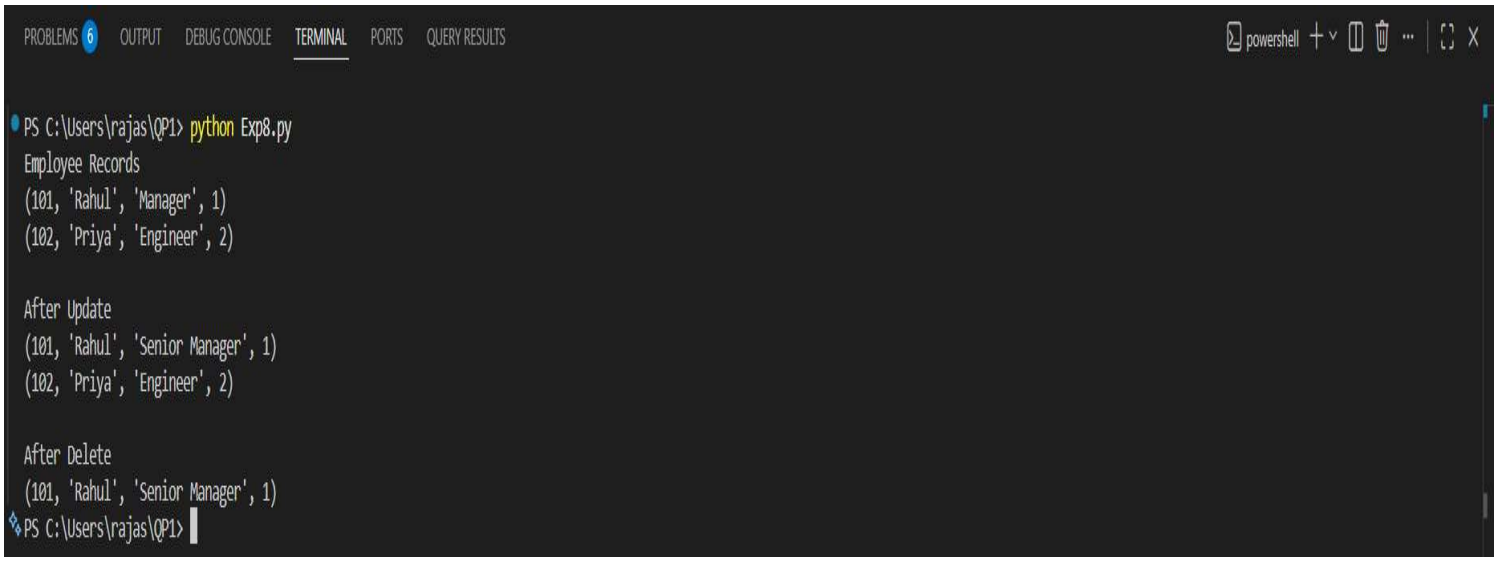
```
cursor.execute("""
CREATE TABLE IF NOT EXISTS Attendance(
    AttendanceID INTEGER PRIMARY KEY,
    EmployeeID INTEGER,
    WorkingDays INTEGER,
    PresentDays INTEGER,
    FOREIGN KEY(EmployeeID) REFERENCES Employee(EmployeeID)
)
""")

# Create Salary Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Salary(
    SalaryID INTEGER PRIMARY KEY,
    EmployeeID INTEGER,
    BasicSalary REAL,
    Bonus REAL,
    FOREIGN KEY(EmployeeID) REFERENCES Employee(EmployeeID)
)
""")

# Create Payroll Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Payroll(
    PayrollID INTEGER PRIMARY KEY,
    EmployeeID INTEGER,
    Tax REAL,
    NetSalary REAL,
```

```
    FOREIGN KEY(EmployeeID) REFERENCES Employee(EmployeeID)
)
""")

# Delete old records
cursor.execute("DELETE FROM Payroll")
cursor.execute("DELETE FROM Salary")
cursor.execute("DELETE FROM Attendance")
cursor.execute("DELETE FROM Employee")
cursor.execute("DELETE FROM Department")

# Insert Department
cursor.execute("INSERT INTO Department VALUES(1,'HR')")
cursor.execute("INSERT INTO Department VALUES(2,'Production')")

# Insert Employee
cursor.execute("INSERT INTO Employee VALUES(101,'Rahul','Manager',1)")
cursor.execute("INSERT INTO Employee VALUES(102,'Priya','Engineer',2)")

# Insert Attendance
cursor.execute("INSERT INTO Attendance VALUES(201,101,30,28)")
cursor.execute("INSERT INTO Attendance VALUES(202,102,30,30)")

# Insert Salary
cursor.execute("INSERT INTO Salary VALUES(301,101,50000,5000)")
cursor.execute("INSERT INTO Salary VALUES(302,102,40000,4000)")

# Insert Payroll
```

```
cursor.execute("INSERT INTO Payroll VALUES(401,101,5000,50000)")
cursor.execute("INSERT INTO Payroll VALUES(402,102,4000,40000)")

conn.commit()

# Read
print("Employee Records")
cursor.execute("SELECT * FROM Employee")
for row in cursor.fetchall():
    print(row)

# Update
cursor.execute("""
UPDATE Employee
SET Designation='Senior Manager'
WHERE EmployeeID=101
""")

conn.commit()

print("\nAfter Update")
cursor.execute("SELECT * FROM Employee")
for row in cursor.fetchall():
    print(row)

# Delete
cursor.execute("""
DELETE FROM Employee
```

```
WHERE EmployeeID=102
""")

conn.commit()

print("\nAfter Delete")
cursor.execute("SELECT * FROM Employee")
for row in cursor.fetchall():
    print(row)

cursor.close()
conn.close()
```

Output:



Scenario 9: Online Learning Management System

An educational technology company provides online courses to students worldwide. Students enroll in courses, watch video lectures, submit assignments, and receive grades. Instructors create courses and manage learning materials. The platform needs a database system to maintain student profiles, instructor details, course catalogs, enrollments, assignments, and assessment results. Administrators

require reports on student progress, course completion rates, and instructor performance. Design a relational database schema that supports these requirements and implement Python programs for managing enrollments and academic records.

Code:

```
import psycopg2

# Connect to PostgreSQL
conn = psycopg2.connect(
    host="localhost",
    database="lms_db",
    user="postgres",
    password="YOUR_PASSWORD",
    port="5432"
)

cursor = conn.cursor()

# Create Student Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Student(
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR(50),
    Email VARCHAR(50)
)
""")

# Create Instructor Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Instructor(
```

```
    InstructorID INT PRIMARY KEY,
    InstructorName VARCHAR(50),
    Department VARCHAR(50)
)
""")

# Create Course Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Course(
    CourseID INT PRIMARY KEY,
    CourseName VARCHAR(50),
    InstructorID INT,
    FOREIGN KEY (InstructorID) REFERENCES Instructor(InstructorID)
)
""")

# Create Enrollment Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Enrollment(
    EnrollmentID INT PRIMARY KEY,
    StudentID INT,
    CourseID INT,
    EnrollmentDate DATE,
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
)
""")
```

```
# Create Assignment Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Assignment(
    AssignmentID INT PRIMARY KEY,
    CourseID INT,
    AssignmentName VARCHAR(100),
    Marks INT,
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
)
""")

# Create Result Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Result(
    ResultID INT PRIMARY KEY,
    StudentID INT,
    AssignmentID INT,
    Grade VARCHAR(5),
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
    FOREIGN KEY (AssignmentID) REFERENCES Assignment(AssignmentID)
)
""")

# Delete old records
cursor.execute("DELETE FROM Result")
cursor.execute("DELETE FROM Enrollment")
cursor.execute("DELETE FROM Assignment")
cursor.execute("DELETE FROM Course")
```

```
cursor.execute("DELETE FROM Instructor")
cursor.execute("DELETE FROM Student")

# Insert Students
cursor.execute("INSERT INTO Student VALUES (101,'Rahul','rahul@gmail.com')")
cursor.execute("INSERT INTO Student VALUES (102,'Priya','priya@gmail.com')")

# Insert Instructors
cursor.execute("INSERT INTO Instructor VALUES (201,'Dr.Ravi','AI')")
cursor.execute("INSERT INTO Instructor VALUES (202,'Dr.Anitha','CSE')")

# Insert Courses
cursor.execute("INSERT INTO Course VALUES (301,'Python Programming',201)")
cursor.execute("INSERT INTO Course VALUES (302,'Database Systems',202)")

# Insert Enrollments
cursor.execute("INSERT INTO Enrollment VALUES (401,101,301,'2026-07-14')")
cursor.execute("INSERT INTO Enrollment VALUES (402,102,302,'2026-07-15')")

# Insert Assignments
cursor.execute("INSERT INTO Assignment VALUES (501,301,'Python Lab',100)")
cursor.execute("INSERT INTO Assignment VALUES (502,302,'SQL Lab',100)")

# Insert Results
cursor.execute("INSERT INTO Result VALUES (601,101,501,'A')")
cursor.execute("INSERT INTO Result VALUES (602,102,502,'B')")

conn.commit()
```

```
# Read
print("Student Records")
cursor.execute("SELECT * FROM Student")
for row in cursor.fetchall():
    print(row)

# Update
cursor.execute("""
UPDATE Student
SET Email='rahul123@gmail.com'
WHERE StudentID=101
""")
conn.commit()

print("\nAfter Update")
cursor.execute("SELECT * FROM Student")
for row in cursor.fetchall():
    print(row)

# Delete (Delete child records first)
cursor.execute("DELETE FROM Result WHERE StudentID=102")
cursor.execute("DELETE FROM Enrollment WHERE StudentID=102")
cursor.execute("DELETE FROM Student WHERE StudentID=102")

conn.commit()

print("\nAfter Delete")
```

```
cursor.execute("SELECT * FROM Student")
for row in cursor.fetchall():
    print(row)

cursor.close()
conn.close()
```

Output:

Scenario 10: Smart Transportation and Ticket Booking System

A transportation company operates buses across multiple cities. Passengers can search routes, book tickets, cancel reservations, and make online payments. The company must maintain information about buses, routes, schedules, drivers, passengers, and ticket bookings. Each bus operates on multiple routes, and passengers may make several bookings over time. Management wants to analyze passenger demand, route utilization, and revenue generation. Design a database system that efficiently stores transportation data and supports ticket booking operations. Use Python SQL libraries to connect to the database and perform CRUD operations for route management, passenger registration, and ticket reservations.

Code:

```
import psycopg2

# Connect to PostgreSQL
```

```
conn = psycopg2.connect(
    host="localhost",
    database="transport_db",
    user="postgres",
    password="raju4216",
    port="5432"
)

cursor = conn.cursor()

# Create Bus Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Bus(
    BusID INT PRIMARY KEY,
    BusName VARCHAR(50),
    BusType VARCHAR(30),
    Capacity INT
)
""")

# Create Route Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Route(
    RouteID INT PRIMARY KEY,
    Source VARCHAR(50),
    Destination VARCHAR(50),
    Distance INT
)
```

```
""")

# Create Driver Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Driver(
    DriverID INT PRIMARY KEY,
    DriverName VARCHAR(50),
    Phone VARCHAR(15)
)
""")

# Create Passenger Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Passenger(
    PassengerID INT PRIMARY KEY,
    PassengerName VARCHAR(50),
    Phone VARCHAR(15)
)
""")

# Create Schedule Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Schedule(
    ScheduleID INT PRIMARY KEY,
    BusID INT,
    RouteID INT,
    DriverID INT,
    JourneyDate DATE,
```

```
    FOREIGN KEY (BusID) REFERENCES Bus(BusID),
    FOREIGN KEY (RouteID) REFERENCES Route(RouteID),
    FOREIGN KEY (DriverID) REFERENCES Driver(DriverID)
)
""")

# Create Booking Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS Booking(
    BookingID INT PRIMARY KEY,
    PassengerID INT,
    ScheduleID INT,
    SeatNo INT,
    Amount DECIMAL(10,2),
    PaymentStatus VARCHAR(20),
    FOREIGN KEY (PassengerID) REFERENCES Passenger(PassengerID),
    FOREIGN KEY (ScheduleID) REFERENCES Schedule(ScheduleID)
)
""")

# Delete old records
cursor.execute("DELETE FROM Booking")
cursor.execute("DELETE FROM Schedule")
cursor.execute("DELETE FROM Passenger")
cursor.execute("DELETE FROM Driver")
cursor.execute("DELETE FROM Route")
cursor.execute("DELETE FROM Bus")
```

```
# Insert Bus
cursor.execute("INSERT INTO Bus VALUES (101,'Express Bus','AC',40)")
cursor.execute("INSERT INTO Bus VALUES (102,'Super Deluxe','Non-AC',50)")

# Insert Route
cursor.execute("INSERT INTO Route VALUES (201,'Chennai','Bangalore',350)")
cursor.execute("INSERT INTO Route VALUES (202,'Hyderabad','Vijayawada',280)")

# Insert Driver
cursor.execute("INSERT INTO Driver VALUES (301,'Ramesh','9876543210')")
cursor.execute("INSERT INTO Driver VALUES (302,'Suresh','9876543211')")

# Insert Passenger
cursor.execute("INSERT INTO Passenger VALUES (401,'Rahul','9999999999')")
cursor.execute("INSERT INTO Passenger VALUES (402,'Priya','8888888888')")

# Insert Schedule
cursor.execute("INSERT INTO Schedule VALUES (501,101,201,301,'2026-07-14')")
cursor.execute("INSERT INTO Schedule VALUES (502,102,202,302,'2026-07-15')")

# Insert Booking
cursor.execute("INSERT INTO Booking VALUES (601,401,501,12,750,'Paid')")
cursor.execute("INSERT INTO Booking VALUES (602,402,502,15,650,'Pending')")

conn.commit()

# Read
print("Passenger Records")
```

```
cursor.execute("SELECT * FROM Passenger")
for row in cursor.fetchall():
    print(row)

# Update
cursor.execute("""
UPDATE Passenger
SET Phone='7777777777'
WHERE PassengerID=401
""")

conn.commit()

print("\nAfter Update")
cursor.execute("SELECT * FROM Passenger")
for row in cursor.fetchall():
    print(row)

# Delete (Delete child records first)
cursor.execute("DELETE FROM Booking WHERE PassengerID=402")
cursor.execute("DELETE FROM Passenger WHERE PassengerID=402")

conn.commit()

print("\nAfter Delete")
cursor.execute("SELECT * FROM Passenger")
for row in cursor.fetchall():
    print(row)
```

cursor.close()

conn.close()

Output: